EXSOIL NSF PROTOTYPE

CTSM container rebuild: summary

A short summary of a container rebuild around the standalone CTSM NEON workflow: what we did, why, and a few open questions.

June 2026 · Steven Wangen, Data Science Institute · srwangen@wisc.edu

This is a short summary of a container rebuild we did around the standalone CTSM/NEON workflow. It covers what we did, why, and a handful of questions where outside input would help us decide whether our workarounds are temporary or long-term. A longer internal report with full technical detail is available on request.

CONTEXT

What we run

We package the CTSM NEON tower workflow inside a Docker container. Researchers pull the image, launch JupyterLab in a browser, and run single-site NEON simulations plus downstream analysis (model-observation comparison and parameter calibration). The container ships the OS, scientific libraries, CTSM source and build system, a Python scientific stack, and JupyterLab.

WHAT WE DID

What we did and why

Our starting point was the `escomp/cesm-lab-neon` image (October 2022). It had three problems we needed to solve:

1. **End-of-life foundation.** CentOS 8 (EOL) and Python 3.7 (EOL), with hundreds of outdated packages.
2. **No Apple Silicon support.** The image was amd64-only. Several team members are on M-series Macs and had to run it under QEMU emulation, which was 2-3x slower and crashed during heavy work (cartopy rendering, model compilation).

3. **A non-functional NEON workflow.** When we tested it, the NEON tower workflow did not run. The original image appears to have grafted NEON support from a pre-release CTSM development branch onto the full CESM 2.2 framework; that graft had broken as components diverged. (It may well have worked when the image was first built in 2022.)

Rather than patch the old image, we rebuilt the stack:

- **Migrated from full CESM 2.2.2 to standalone CTSM 5.4.043**, which includes the NEON workflow natively. This dropped ~3-4 GB of unused atmosphere/ocean/ice/wave source and gave us a supported, single-site land configuration. This alone resolved most of the original build issues.
- **Moved to Ubuntu 24.04** as the base (supported, multi-arch).
- **Replaced in-image source compilation of MPICH/HDF5/NetCDF-C/NetCDF-Fortran/PNetCDF with conda-forge pre-built binaries.** The old image compiled these five libraries from source on every build (30-45 min, architecture-specific). conda-forge ships builds for both amd64 and arm64, which cut build time to ~5 min and removed the most architecture-dependent part of the build.
- **Published a true multi-arch image** (amd64 + arm64 in one manifest list). `docker pull` selects the right architecture automatically; downstream commands are unchanged. arm64 now runs natively.
- **Trimmed the Python environment** from ~400 packages to 35 direct dependencies, with conda-lock lockfiles pinned per architecture for reproducibility.

CTSM Fortran compilation still happens at runtime via `case.build` (we ship the toolchain), confirmed working natively on arm64 in ~100 seconds.

Moving to CTSM 5.4 also brought the individual components forward, so simulations now use current science rather than a frozen 2022 snapshot:

COMPONENT	BEFORE (V1)	AFTER (V2)
Land model	CLM ~5.x (dev-branch graft, no official release)	CLM 6.0 (improved soil biogeochemistry, updated PFTs, soil hydrology)
Build/case system	CIME (CESM 2.2 era)	CIME 6.1
Forcing scenarios	CLM 5.x-era CMIP6 datasets	Updated CMIP6/CMIP7 (nitrogen deposition, aerosols, ozone, fire-model population density)
NEON tower data	Through ~2022	Through ~September 2024
Python	3.7 (EOL)	3.13

WORKAROUNDS

A couple of workarounds we'd like your read on

Two of our changes are container-level workarounds rather than upstream fixes, and it would help to know whether these are known issues:

- **`MPILIB=mpi-serial` conflicts with conda-forge MPICH.** The NEON usermods default to `mpi-serial` (CIME's serial stub). In a conda-forge environment the real MPICH shared libraries are always on the linker path, and the two conflict at runtime. We patch the NEON usermods in our Dockerfile to drop the `mpi-serial` override so cases use real MPICH. This has to be reapplied on every CTSM upgrade.
- **Standard arm64 / GCC-strictness compile fixes** in our machine config: removing `-DHAVE_NANOTIME` (x86-only `rdtsc` in GPTL), adding `-fallow-argument-mismatch` and `-fallow-invalid-boz` (GCC 10+), `-D_FillValue=NC_FillValue` (NetCDF-C rename), and setting `ESMFMKFILE`. These are localized and stable, but if any belong upstream we'd be glad to contribute them back.

QUESTIONS

Open questions

On the development tag

We pinned to **CTSM 5.4.043, a development tag on `main`, not an official release.** We did this because the most recent official release (ctsm5.4.002, December 2025) shipped before the data-server config was updated, so it still points at servers where the input data no longer exists. 5.4.043 was the earliest tag we found that resolves data correctly against GDEX.

When can we expect a stable CTSM 5.4.x release with working GDEX server config that we can pin to instead of tracking a dev tag?

On data hosting / GDEX

The new GDEX endpoint (`osdf-data.gdex.ucar.edu`) uses a 3-hop redirect chain that intermittently returns empty responses or times out. We work around it with a pre-download script (5 retries per file, fallback to SVN and FTP), which gets us to 100% success across 31 files but still takes 10-15 minutes on first setup, with individual stalls up to ~5 minutes.

Is the GDEX CDN reliability being actively addressed, or should we treat this as the new normal and design around it (e.g., cache the ~6 GB of input data ourselves)?

On the mpi-serial / conda-forge conflict

Is the `MPILIB=mpi-serial` vs. real-MPI-on-path conflict (described above) a known issue when running CTSM in conda-based environments? Is there a recommended pattern we should follow instead of patching the usermods?

On NEON tower forcing coverage

Tower forcing currently runs through ~September 2024 for most sites (84 monthly files from January 2018). The last three months of 2024 (Oct-Dec) are not yet on NEON's portal.

Are those months expected to be published, and on what rough timeline?

We're happy to file formal GitHub issues against ESCOMP/CTSM for any of these. We held off pending a sense of whether they're already known. The standalone CTSM 5.4 NEON setup made this rebuild dramatically simpler than the path we started on.